Code Refactoring

Improving Code Quality and Maintainability

What is Code Refactoring?

- Code refactoring is the process of **restructuring existing code** without changing its **external behavior**.

- Example (Before):

```
public int add(int a, int b) { return a+b; }
```

- Example (After):

```
public int addNumbers(int firstNumber, int secondNumber)
{
return firstNumber + secondNumber;
}
```

Why is Code Refactoring Important?

- Improves code readability
- Reduces technical debt
- Enhances performance
- Simplifies debugging and testing
- Helps in DevOps automation

Basic Principles of Code Refactoring

- Keep the behavior of application the same.
- Small, incremental code changes, avoid large changes, it may break the code.
- Continuous testing after refactoring.
- Follow clean code principles
- Use automation tools (Eclipse, SonarQube)

When to Refactor?

- Before adding new features
- When fixing bugs
- When code smells are detected
- Before deployment in a DevOps pipeline

Common Code Smells Indicating Need for Refactoring

- Duplicated Code
- Long Methods
- Large Classes
- Too Many Parameters
- High Coupling When one class **depends too much** on another class

Refactoring Approach - Extract Method

- Example (Before):

```
public void printUserDetails(String name, int age) {
    System.out.println("User Name: " + name);
    System.out.println("User Age: " + age);
}
```

- Example (After):

```
private void printName(String name)
{ System.out.println("User Name: " + name); }
private void printAge(int age)
{ System.out.println("User Age: " + age); }
```

Refactoring Approach - Rename Variables

- Example (Before):

```
int d = 100;
```

- Example (After):

```
int accountBalance = 100;
```

Refactoring Approach - Replace Magic Numbers

- Example (Before):

```
public double calculateCircleArea(double radius)
{ return 3.1416 * radius * radius; }
```

- Example (After):

```
public static final double PI = 3.1416;
public double calculateCircleArea(double radius)
{ return PI * radius * radius; }
```

Refactoring Approach - Replace Switch with Polymorphism

Before

```
public double getDiscount(String customerType) {
    switch(customerType) {
        case "Regular": return 10.0;
        case "VIP": return 20.0;
        default: return 5.0;
    }
}
```

After

```
interface Discount {
    double getDiscount();
}
class RegularCustomer implements Discount {
    public double getDiscount() { return 10.0; }
}
class VIPCustomer implements Discount {
    public double getDiscount() { return 20.0; }
}
```

Refactoring Approach - Replace Loops with Streams

Before

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
for (String name : names) {
    System.out.println(name.toUpperCase());
}
```

After

```
names.stream().map(String::toUpperCase).forEach(System.out::println);
```

names.stream() → Converts the list into a stream.
map(String::toUpperCase) → Transforms each element by converting it to uppercase.
forEach(System.out::println) → Iterates over each transformed element and prints it.

Tools for Code Refactoring in Eclipse IDE

- Eclipse Refactoring Menu
- SonarQube for Code Quality Analysis
- Checkstyle & PMD for Code Smells
- JUnit for Testing
- Visual Studio Code + Copilot

Automating Code Refactoring with DevOps

- Static Code Analysis using (SonarQube, Checkstyle)
- Continuous Integration (CI/CD) using (GitHub Actions, Jenkins)
- Automated Testing using (JUnit, Selenium)

Testing After Refactoring

- **Unit Testing (JUnit)**

  Testing individual units of code, typically functions or methods, to ensure they work as expected.

- **Integration Testing**

  Testing the interaction between integrated units/modules to detect interface defects.

- **Performance Testing**

  Evaluating the speed, responsiveness, and stability of a system under load.

Refactoring vs Rewriting Code

Rewriting means developing a new code to replace the existing one.

Refactoring means improving the existing code without changing its external behavior.

Feature	Refactoring	Rewriting
Time	Fast	Slow
Risk	Low	High
Testing Required	Minimal	Extensive

Challenges in Code Refactoring

- **Understanding Legacy Code** can be complex and poorly documented.
- Changes might **introduce new issues** such as Bugs.
- Difficult to find **time** amid other development tasks.
- **Extensive testing** needed to ensure everything works.
- Need to select the **right tools** to support refactoring.

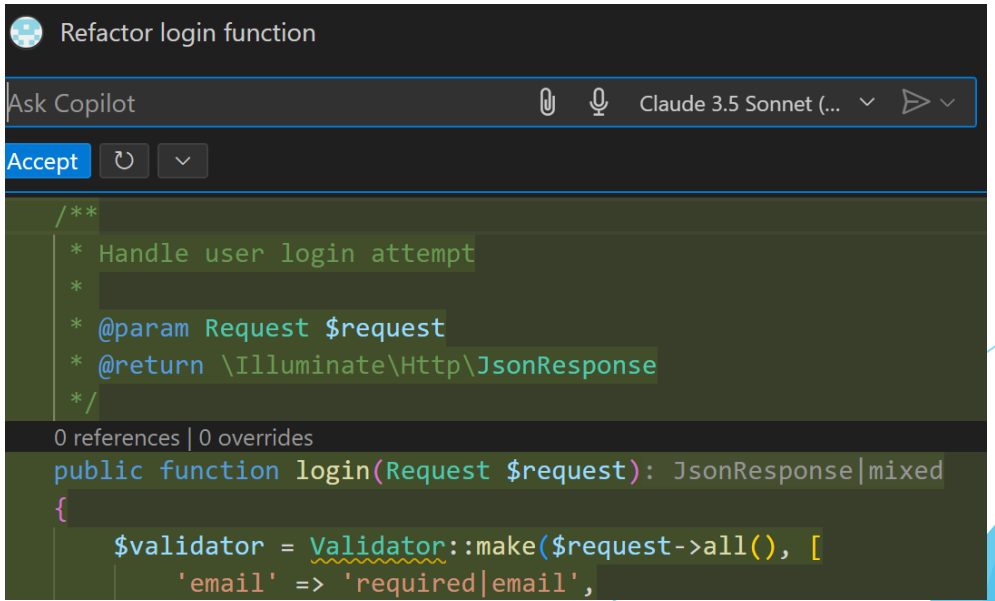
Future of Code Refactoring with AI

- AI-based refactoring suggestions

```
0 references
public function setPasswordAttribute($password)
```

- Automated Code Generation and Review Tools

Copilot



Best Practices in Code Refactoring

- Keep methods short
- Use meaningful names
- Apply SOLID principles

SOLID

- **S - Single Responsibility Principle (SRP)**
  A class should have **only one reason to change.**

```java
class Report {
    public void generateReport() {
        System.out.println("Generating report...");
    }

    public void saveToDatabase() {
        System.out.println("Saving report to database...");
    }
}
```



```java
class ReportGenerator {
    public void generateReport() {
        System.out.println("Generating report...");
    }
}

class ReportSaver {
    public void saveToDatabase() {
        System.out.println("Saving report to database...");
    }
}
```

SOLID

- **O - Open/Closed Principle (OCP)**
  Software components should be **open for extension but closed for modification.**

```
class DiscountCalculator {
    public double calcDiscount(String customerType, double price) {
        if (customerType.equals("Regular")) {
            return price * 0.10;
        } else if (customerType.equals("VIP")) {
            return price * 0.20;
        }
        return 0;
    }
}
```

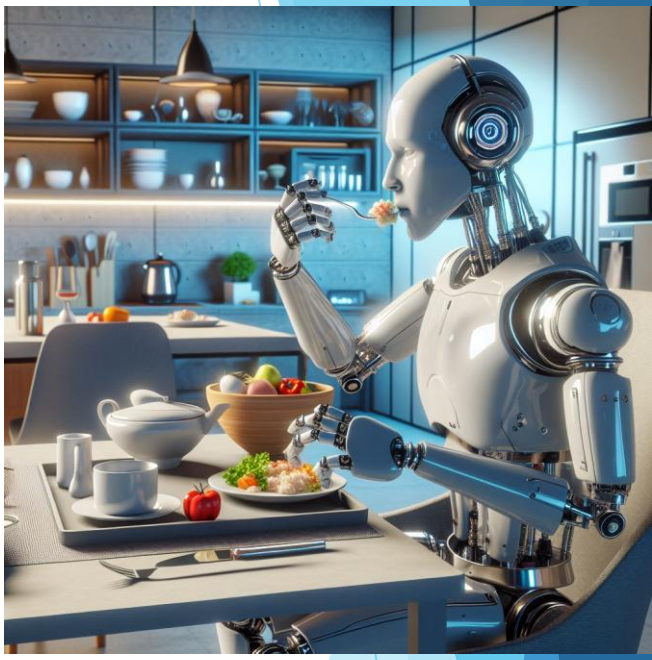
```
interface Discount {
    double applyDiscount(double price);
}

class RegularDiscount implements Discount
class VIPDiscount implements Discount
class Members implements Discount
```

Can extend class to include members

SOLID

- **L - Liskov Substitution Principle (LSP)** Subtypes should be **replaceable** for their base types **without breaking the program.**
- LSP means that if a program works correctly with a parent class, it should also work correctly with any of its subclasses without modification.
- Example: Robot cannot eat, but it inherits eat() from Worker and throws an exception.



```
class Worker {
    void work() { System.out.println("Working..."); }
    void eat() { System.out.println("Eating..."); }
}

class Robot extends Worker {
      void eat() { throw new UnsupportedOperationException("Robots don't eat!");
}
}
```

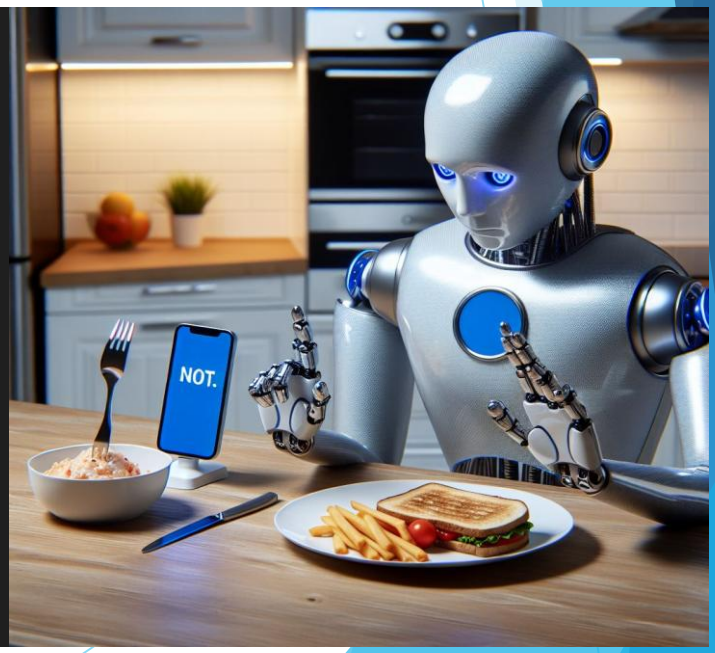
SOLID

- **L - Liskov Substitution Principle (LSP)**
  Subtypes should be **replaceable** for their base types **without breaking the program.**
- Example: Separate eat() behavior using an interface

```
interface Workable {
    void work();
}

interface Eatable {
    void eat();
}

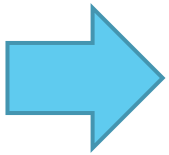
Class Human implements Workable, Eatable
class Robot implements Worker
```



SOLID

- **I - Interface Segregation Principle (ISP)**
  A class should **not be forced** to implement methods it **does not use.**

```
interface Worker {
    void work();
    void eat();
}
Class Human implements Worker
class Robot implements Worker
```



```
interface Workable {
    void work();
}

interface Eatable {
    void eat();
}

Class Human implements Workable, Eatable
class Robot implements Worker
```

SOLID

- **D - Dependency Inversion Principle (DIP)**
  High-level modules should **not depend on low-level modules.** Both should **depend on abstractions.**

```
class MySQLDatabase { //Low level
    public void connect() {
        System.out.println("Connecting to MySQL...");
    }
}
class Application {// High level
    private MySQLDatabase database = new MySQLDatabase();

    public void start() {
        database.connect();
    }}
```

Problem: Application uses MySQLDatabase. If we switch to another database, we must modify Application.

SOLID

- D - Dependency Inversion Principle (DIP)

```
// Abstraction (Interface)
interface Database {
    void connect();
}
// Low-Level Module (Implements Abstraction)
class MySQLDatabase implements Database {
    public void connect() { System.out.println("Connected to MySQL");
}
}
// High-Level Module (Depends on Abstraction)
class Application {
    private Database db;
}
```

Solution: Application depends on Database (abstraction) instead of MySQLDatabase

Summary

- Refactoring improves code without changing functionality.
- Common Techniques: Remove duplicates, use streams, improve class design.
- SOLID Principles
- DevOps & Quality: CI/CD, automated testing